C Programming

Lecture Notes

Recursion in C Programming

A Beginner-Friendly, Step-by-Step Classroom Handout

Theory • Examples • Dry Runs • Practice

1. Introduction to Recursion

Welcome, students! Today we are going to explore one of the most elegant and powerful concepts in computer programming — Recursion. Don't worry if the word sounds unfamiliar; by the end of these notes you will feel completely comfortable using it in your own programs.

What is Recursion — in Simple Words

In everyday life, sometimes the best way to solve a big problem is to break it into a smaller version of the same problem. You keep breaking it down until the problem becomes so tiny that the answer is obvious. That idea — solving a problem by reducing it to a smaller copy of itself — is exactly what recursion is.

In programming terms:

- Recursion is a technique where a function calls itself to solve a smaller version of the same problem.
- The function keeps calling itself until it reaches a simple, known answer (called the base case).
- Once the simple answer is found, all the waiting calls return their results one by one.

Why Do We Need Recursion?

You might be wondering: "Can't loops do the same thing?" Yes — in most cases, what recursion can do, a loop can also do. But consider these situations where recursion shines:

- Tree structures (file systems, family trees) — naturally recursive.
- Divide-and-Conquer algorithms (Merge Sort, Quick Sort).
- Mathematical problems like factorials, Fibonacci, Tower of Hanoi.
- When the recursive solution is significantly shorter and cleaner than the loop version.

Recursion vs Loops — Motivation

Let's compare the mental model:

Loops (Iterative)	Recursion
You explicitly manage a counter or condition.	The function manages itself by calling itself.
You repeat a block of code.	You repeat function calls.
State is stored in variables.	State is stored in the call stack (automatically).
Generally faster in execution.	Slightly slower due to function call overhead.
Better for simple, sequential repetition.	Better for problems that are naturally self-similar.

KEY INSIGHT: Recursion is not a replacement for loops — it is a different tool. Great programmers know when to use each one.

2. What is Recursion?

Formal Definition

Recursion is a programming technique in which a function directly or indirectly calls itself as part of its own definition, with each call working on a progressively simpler version of the original problem, until a terminating condition (the base case) is reached.

A Function That Calls Itself

Here is the simplest possible recursive function in C — it just counts down:

```
void countDown(int n) {
    if (n == 0) {                      // base case: stop here
        printf("Go!\n");
        return;
    }
    printf("%d...\n", n);             // print current number
    countDown(n - 1);                  // call itself with a smaller value
}
```

Notice that countDown calls countDown inside its own body. That is recursion. Each time it calls itself, n gets smaller, until it reaches 0 and stops.

Real-Life Analogies

Understanding recursion through analogies makes it much easier to grasp:

Analogy 1 — Facing Mirrors

Stand between two mirrors facing each other. You see a reflection of yourself, which contains another reflection, which contains another, going on and on. Each reflection is a smaller version of the original scene. Recursion works the same way — each function call is a "smaller reflection" of the original problem.

Analogy 2 — Nested Boxes

Imagine a box containing a smaller box, which contains an even smaller box, which contains... a coin (the treasure). To find the coin, you open boxes one by one until you reach the innermost box. The base case is finding the coin; every other step is the recursive case of "open the box and look inside."

Analogy 3 — Staircase

How many steps are there from the 1st floor to the 5th floor? Answer: 1 step + (steps from 2nd floor to 5th floor). And steps from 2nd to 5th = 1 step + steps from 3rd to 5th... and so on. You reduce the problem one step at a time. That is recursion!

3. Structure of a Recursive Function

Every correct recursive function has exactly two essential components. Missing either one causes problems. Let's study them carefully.

Part 1 — The Base Case

The base case is the stopping condition. It is the simplest version of the problem that can be answered directly without any further recursion.

- It tells the function: "Stop calling yourself. I know the answer!"
- Without a base case, the function will call itself forever.
- There can be one or more base cases depending on the problem.

Part 2 — The Recursive Case

The recursive case is the part where the function calls itself. It must always move closer to the base case — otherwise, the recursion will never end.

- It says: "I don't know the full answer yet, but I can break this into a smaller piece."
- It calls the same function with a simpler or smaller argument.
- Each recursive call MUST progress toward the base case.

Template of a Recursive Function

```
return_type functionName(parameters) {

    // ── BASE CASE ──────────────────────────────────────
    if (simplest_condition) {
        return known_answer;   // no recursive call here
    }

    // ── RECURSIVE CASE ─────────────────────────────────
    // Do some work, then call itself with a smaller problem
    return functionName(smaller_problem);
}
```

What Happens If the Base Case is Missing?

This is a very important question. If you forget the base case, the function will call itself endlessly. This is called infinite recursion. Let's see what happens:

```
// DANGER: No base case — DO NOT run this!
```

```
void countdown(int n) {
    printf("%d\n", n);
    countdown(n - 1);   // calls forever — never stops!
}
```

Each function call takes up space in a memory area called the call stack. If the recursion never ends, the stack fills up completely, causing a Stack Overflow error and crashing your program.

RULE TO MEMORIZE: Every recursive function MUST have a base case, AND the recursive call MUST move the input toward that base case. Both are non-negotiable.

4. Execution Flow of Recursion

Step-by-Step Execution

Let's trace through what actually happens inside the computer when a recursive function is called. We'll use a simple example: calculating factorial(3).

Here is the factorial function:

```
int factorial(int n) {
    if (n == 0)                          // base case
        return 1;
    return n * factorial(n - 1);         // recursive case
}
```

When you call factorial(3), here is the exact sequence:

1. factorial(3) is called. n is not 0, so it computes 3 * factorial(2) and PAUSES, waiting for the result.
2. factorial(2) is called. n is not 0, so it computes 2 * factorial(1) and PAUSES.
3. factorial(1) is called. n is not 0, so it computes 1 * factorial(0) and PAUSES.
4. factorial(0) is called. n IS 0 — BASE CASE! Returns 1 immediately.
5. factorial(1) resumes: 1 * 1 = 1. Returns 1.
6. factorial(2) resumes: 2 * 1 = 2. Returns 2.
7. factorial(3) resumes: 3 * 2 = 6. Returns 6.

The Function Call Stack

The call stack is a special area of memory where the computer keeps track of all function calls that are waiting to finish. Think of it as a stack of plates:

- Every time a function is called, a new plate (called a stack frame) is placed on top.
- Each frame stores: the local variables, the parameters, and where to return when done.
- When a function returns, its plate is removed from the top.
- The function below it (the one that was waiting) resumes.

Visual representation of the call stack for factorial(3):

```
CALL PHASE (stack grows upward)        RETURN PHASE (stack shrinks)
──────────────────────────────         ────────────────────────────────
[ factorial(0) ] ← top                 factorial(0) returns 1
[ factorial(1) ]                       factorial(1) returns 1*1 = 1
[ factorial(2) ]                       factorial(2) returns 2*1 = 2
[ factorial(3) ] ← bottom              factorial(3) returns 3*2 = 6
```

```
[ main()        ]                        main() receives 6
```

THINK OF IT THIS WAY: Recursion has two phases — the WINDING phase (calls going deeper) and the UNWINDING phase (values returning back up). The real answer is built during the unwinding.

5. Basic Example — Factorial of a Number (Full Dry Run)

The Problem

Factorial of a number n (written as n!) is defined as:

```
  n! = n * (n-1) * (n-2) * ... * 2 * 1
  0! = 1   (by definition)

  Example:  5! = 5 * 4 * 3 * 2 * 1 = 120
```

Full C Code

```
#include <stdio.h>

/* Function: factorial
   Purpose : Returns n! (n factorial) using recursion
   Input   : A non-negative integer n
   Output  : The factorial value               */
int factorial(int n) {
    // BASE CASE: factorial of 0 is 1
    if (n == 0) {
        return 1;
    }

    // RECURSIVE CASE: n! = n * (n-1)!
    return n * factorial(n - 1);
}

int main() {
    int num = 5;
    int result = factorial(num);
    printf("%d! = %d\n", num, result);  // Output: 5! = 120
    return 0;
}
```

Step-by-Step Dry Run for factorial(5)

Let's trace every single call:

```
factorial(5)
  └─ 5 * factorial(4)
           └─ 4 * factorial(3)
                    └─ 3 * factorial(2)
                             └─ 2 * factorial(1)
                                      └─ 1 * factorial(0)
```

```
                                                    └─ returns 1  ← BASE CASE


Now values return back up:
  factorial(1) = 1 * 1 = 1
  factorial(2) = 2 * 1 = 2
  factorial(3) = 3 * 2 = 6
  factorial(4) = 4 * 6 = 24
  factorial(5) = 5 * 24 = 120   ✓
```

OBSERVE: Notice how no actual multiplication happens during the call phase. All the multiplication happens on the way back up (return phase). This is a key insight about recursion!

6. More Examples

Example 1 — Sum of First N Natural Numbers

Find the sum: 1 + 2 + 3 + ... + n

Recursive insight:

- sum(n) = n + sum(n-1)
- sum(0) = 0 (base case)

```
#include <stdio.h>

/* Returns the sum of first n natural numbers */
int sum(int n) {
    // BASE CASE
    if (n == 0) {
        return 0;
    }

    // RECURSIVE CASE: add n to the sum of (n-1) numbers
    return n + sum(n - 1);
}

int main() {
    printf("Sum of 1 to 5 = %d\n", sum(5));  // Output: 15
    return 0;
}
```

Explanation:

- sum(5) = 5 + sum(4) = 5 + 4 + sum(3) = 5+4+3+2+1+0 = 15
- Base case: sum(0) = 0 stops the recursion.
- Each call reduces n by 1, guaranteeing we reach 0.

Example 2 — Fibonacci Sequence

The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
Each number is the sum of the two before it.

Recursive definition:

- fib(0) = 0 (base case 1)
- fib(1) = 1 (base case 2)
- fib(n) = fib(n-1) + fib(n-2) (recursive case)

```c
#include <stdio.h>

/* Returns the nth Fibonacci number */
int fib(int n) {
    // BASE CASES: first two Fibonacci numbers
    if (n == 0) return 0;
    if (n == 1) return 1;

    // RECURSIVE CASE: sum of previous two
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int i;
    printf("First 8 Fibonacci numbers:\n");
    for (i = 0; i < 8; i++) {
        printf("fib(%d) = %d\n", i, fib(i));
    }
    return 0;
}
```

Explanation:

- This function has TWO base cases because both fib(0) and fib(1) are known.
- For n=5: fib(5) = fib(4)+fib(3) = (fib(3)+fib(2)) + (fib(2)+fib(1))... and so on.
- This creates a tree of recursive calls (we will see this in Section 7).

IMPORTANT NOTE: Recursive Fibonacci is elegant but slow for large n. It recalculates the same values many times. For large n, use an iterative approach or dynamic programming.

Example 3 — Print Numbers from 1 to N

Print: 1, 2, 3, ..., n using recursion.

```c
#include <stdio.h>

/* Prints numbers from 1 to n */
void printNumbers(int n) {
    // BASE CASE: nothing to print if n is 0
    if (n == 0) {
        return;
    }

    // RECURSIVE CASE: first print 1 to (n-1), then print n
    printNumbers(n - 1);   // go deeper first
    printf("%d ", n);      // print on the way back
}
```

```
int main() {
    printNumbers(5);    // Output: 1 2 3 4 5
    printf("\n");
    return 0;
}
```

Explanation:

- The trick here is that printf comes AFTER the recursive call.
- So the function dives all the way down to printNumbers(0) first, then prints on the way back up.
- Result: 1 2 3 4 5 — the correct ascending order.
- If you move printf BEFORE the recursive call, you get 5 4 3 2 1 (reverse order). Try it!

7. Recursion Tree — Understanding Calls Visually

What is a Recursion Tree?

A recursion tree is a diagram that shows all the function calls made during recursion. Each node in the tree represents one function call. The children of a node are the calls that node makes. This is especially useful for functions like Fibonacci that make multiple recursive calls.

Recursion Tree for fib(4)

```
                      fib(4)
                    /        \
               fib(3)         fib(2)
              /     \         /     \
          fib(2)  fib(1)  fib(1)  fib(0)
          /    \    |       |       |
      fib(1) fib(0) 1       1       0
        |      |
        1      0

Total calls made: 9
Result: 1+0+1+1+0 = 3   ✓  (fib(4) = 3)
```

Reading the tree:

- Start from the top (root): fib(4).
- Each node splits into two children: fib(n-1) and fib(n-2).
- Leaf nodes (bottom) are base cases: they return 0 or 1.
- Values are summed on the way back up to the root.

Recursion Tree for factorial(4)

Factorial only makes one recursive call, so it forms a straight line (not a tree):

```
factorial(4)
     |
factorial(3)
     |
factorial(2)
     |
factorial(1)
     |
factorial(0)  → returns 1

This is called a LINEAR recursion.
Fibonacci-style is called TREE recursion.
```

TERMINOLOGY: When a function calls itself only once, it is called Linear Recursion. When it calls itself more than once (like Fibonacci), it is called Tree Recursion. Tree recursion is more expensive because the number of calls grows exponentially.

8. Advantages of Recursion

Now that we understand how recursion works, let's appreciate why programmers love it:

1. Code Simplicity and Elegance

Recursive solutions are often much shorter and easier to read than their iterative counterparts. Compare the recursive and iterative factorial:

```
// RECURSIVE — clean and reads like math
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

// ITERATIVE — requires extra variables and a loop
int factorial_iter(int n) {
    int result = 1;
    int i;
    for (i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

The recursive version is shorter, directly mirrors the mathematical definition, and requires no extra variables.

2. Naturally Suits Divide-and-Conquer Problems

- Merge Sort, Quick Sort, Binary Search — all naturally recursive.
- Tree traversals (Inorder, Preorder, Postorder) — almost impossible without recursion.
- Tower of Hanoi — the recursive solution is 3 lines; iterative needs complex logic.
- Graph algorithms (DFS — Depth First Search) — elegantly expressed with recursion.

3. Easier to Prove Correctness

Recursive solutions are closer to mathematical induction, making it easier to formally verify that they are correct. This matters a lot in computer science theory and algorithm design.

9. Disadvantages of Recursion

Recursion is not always the best tool. Here are the important limitations you must keep in mind:

1. Memory Usage — The Call Stack

Every recursive call adds a new frame onto the call stack. For deep recursion (say, n = 10000), there could be 10,000 stack frames all sitting in memory at the same time. Each frame uses memory for local variables and return addresses.

- Deep recursion uses significantly more memory than an equivalent loop.
- The call stack size is limited — typically 1 MB to 8 MB on most systems.

2. Slower Than Iteration in Many Cases

Each function call has overhead: the CPU must save the current state, jump to the function, and restore the state on return. For recursive Fibonacci(50), millions of calls are made — many repeating the same calculations.

- Function call overhead adds up for deep or branching recursion.
- Iterative loops have no such overhead.

3. Risk of Stack Overflow

If there is no base case, or if the base case is never reached (due to a bug), the recursion continues indefinitely, filling up the call stack completely. The operating system then terminates your program with a Stack Overflow error.

```
// This will cause a Stack Overflow!
int badFunction(int n) {
    return n + badFunction(n - 1);  // No base case!
}
```

4. Harder to Debug

When recursion goes wrong, tracing through many levels of function calls is much harder than stepping through a simple loop. Stack traces from crashes can be dozens of lines long.

10. Recursion vs Looping — Clear Comparison

Both recursion and loops achieve repetition, but they differ in important ways. Here is a comprehensive comparison:

Feature	Recursion	Looping (Iterative)
Mechanism	Function calls itself	Uses for/while/do-while
Termination	Base case	Loop condition
State storage	Call stack (automatic)	Variables (explicit)
Memory	Higher (stack frames)	Lower (just variables)
Speed	Slower (call overhead)	Faster
Code length	Often shorter	Can be longer
Readability	High for recursive problems	High for sequential problems
Risk	Stack overflow	Infinite loop
Best for	Trees, graphs, divide & conquer	Arrays, counters, simple repeat

When to Use Recursion

- The problem is naturally defined recursively (trees, graphs, fractals).
- The recursive solution is significantly simpler than the iterative one.
- You are implementing algorithms like Merge Sort, Quick Sort, DFS, BFS.
- The input size is moderate (no risk of deep stack overflow).

When to Use Loops

- The problem involves simple repetition over a sequence.
- Performance is critical (loops are faster).
- The input size is very large (risk of stack overflow with recursion).
- Memory is limited.

GOLDEN RULE: If you can write the recursive and iterative solutions with similar simplicity, prefer the iterative one. Use recursion only when it makes the code meaningfully clearer.

11. Common Mistakes in Recursion

Even experienced programmers make mistakes with recursion. Here are the most common pitfalls — study them carefully so you can avoid them!

Mistake 1 — Missing the Base Case

The most common beginner mistake. Without a base case, the function never stops.

```
// WRONG: No base case
int factorial(int n) {
    return n * factorial(n - 1);   // Will run forever → Stack Overflow
}

// CORRECT: Base case added
int factorial(int n) {
    if (n == 0) return 1;          // Base case
    return n * factorial(n - 1);
}
```

Mistake 2 — Not Moving Toward the Base Case

The recursive call must always make the problem simpler. If it doesn't, you still get infinite recursion even with a base case.

```
// WRONG: Calls factorial(n+1) — moving AWAY from base case!
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n + 1);   // n increases, never reaches 0
}

// CORRECT: Call factorial(n-1) — moving TOWARD base case
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);   // n decreases toward 0
}
```

Mistake 3 — Wrong Base Case Condition

Even if you have a base case, it must be the right condition.

```
// WRONG: Base case checks n == 1 but we also pass n = 0
int sum(int n) {
    if (n == 1) return 1;          // Misses the case when n = 0
    return n + sum(n - 1);
```

```
}
// sum(0) will call sum(-1), sum(-2)... infinitely!

// CORRECT: Check n <= 0 or n == 0
int sum(int n) {
    if (n <= 0) return 0;          // Handles all edge cases safely
    return n + sum(n - 1);
}
```

Mistake 4 — Forgetting to Return the Recursive Call's Value

```
// WRONG: Return value is ignored
int factorial(int n) {
    if (n == 0) return 1;
    factorial(n - 1);              // Called but result is THROWN AWAY
}

// CORRECT: Return the recursive call's result
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);   // Result is used
}
```

12. Practice Problems

Now it's time to apply what you've learned! Solve the following problems using recursion. For each problem, clearly identify the base case and the recursive case before writing code.

Problem 1 — Print Reverse of a Number

Write a recursive function that prints the digits of a number in reverse order.
Example: reverse(12345) prints 5 4 3 2 1

Hint: The last digit is n % 10. Remove it with n / 10.

```
#include <stdio.h>

/* Prints digits of n in reverse order */
void reverse(int n) {
    // BASE CASE: nothing to print if n is 0
    if (n == 0) return;

    printf("%d ", n % 10);   // print last digit first
    reverse(n / 10);          // recurse on remaining digits
}

int main() {
    reverse(12345);   // Output: 5 4 3 2 1
    return 0;
}
```

Problem 2 — Power Function (x raised to n)

Write a recursive function power(x, n) that computes x^n.
Example: power(2, 5) = 32

Hint: x^n = x * x^(n-1), and x^0 = 1.

```
#include <stdio.h>

/* Returns x raised to the power n */
int power(int x, int n) {
    // BASE CASE: anything to the power 0 is 1
    if (n == 0) return 1;

    // RECURSIVE CASE: x^n = x * x^(n-1)
    return x * power(x, n - 1);
}
```

```
int main() {
    printf("2^5 = %d\n", power(2, 5));   // Output: 32
    printf("3^4 = %d\n", power(3, 4));   // Output: 81
    return 0;
}
```

Problem 3 — Check if a String is a Palindrome

Write a recursive function that checks if a string reads the same forwards and backwards.
Example: "madam" is a palindrome; "hello" is not.

```
#include <stdio.h>
#include <string.h>

/* Returns 1 if s[left..right] is a palindrome, else 0 */
int isPalindrome(char s[], int left, int right) {
    // BASE CASE 1: single character is always a palindrome
    if (left >= right) return 1;

    // If outer characters don't match — not a palindrome
    if (s[left] != s[right]) return 0;

    // RECURSIVE CASE: check the inner substring
    return isPalindrome(s, left + 1, right - 1);
}

int main() {
    char word[] = "madam";
    int len = strlen(word);
    if (isPalindrome(word, 0, len - 1))
        printf("%s is a palindrome\n", word);
    else
        printf("%s is NOT a palindrome\n", word);
    return 0;
}
```

Problem 4 — Count Digits in a Number

Write a recursive function that counts the number of digits in a positive integer.
Example: countDigits(12345) = 5

```
#include <stdio.h>

/* Counts the number of digits in n */
int countDigits(int n) {
```

```
    // BASE CASE: single digit number
    if (n < 10) return 1;

    // RECURSIVE CASE: remove last digit, count rest
    return 1 + countDigits(n / 10);
}

int main() {
    printf("%d\n", countDigits(12345));   // Output: 5
    printf("%d\n", countDigits(7));       // Output: 1
    return 0;
}
```

Additional Exercises (Try on Your Own)

Here are some more problems to practice — solutions not provided. Challenge yourself!

8. GCD (Greatest Common Divisor): gcd(a, b) = gcd(b, a % b), and gcd(a, 0) = a.
9. Sum of digits of a number: sumDigits(123) = 1+2+3 = 6.
10. Binary search using recursion on a sorted array.
11. Tower of Hanoi: move n disks from source peg to destination peg.
12. Print all elements of an array recursively.
13. Find the maximum element in an array using recursion.

13. Thought Question — Stop and Think!

Think About This!

Consider this simple recursive function:

```
void mystery(int n) {
    if (n <= 0) return;
    mystery(n - 1);
    mystery(n - 1);
}
```

Question 1: How many times is mystery() called in total when you call mystery(4)?

Question 2: How does the number of calls relate to n? Can you express it as a formula?

Question 3: Draw the recursion tree for mystery(3). How many levels deep does it go?

Hint: Think about binary trees and powers of 2...

Quick Summary — Key Points to Remember

- ✓ Recursion = a function calling itself to solve a smaller version of the same problem.
- ✓ Every recursive function MUST have a base case.
- ✓ Every recursive call MUST move closer to the base case.
- ✓ Recursion uses the call stack — deep recursion uses more memory.
- ✓ Two phases: winding (calls going down) and unwinding (values returning up).
- ✓ Missing base case → infinite recursion → Stack Overflow crash.
- ✓ Use recursion when the problem is naturally self-similar; use loops for simple repetition.